

## Test Plan (Restful Booker)

Created by Pramod Dutta

TheTestingAcademy.com

### Objective

The objective of this test plan is to formally verify the RESTful Booker API, where we have created booking, updated booking, deleted booking with authentication, and authentication token. It basically contains a lot of test cases. We need to find all these bugs. That's the overall objective. We are going to use the manual testing to do that validation activity. At the end of the day, we need automation also for the same by using REST Assured framework.

URL: <https://restful-booker.herokuapp.com/apidoc/index.html>

### Scope

Scope of Test Plan for Restful Booker API:

#### 1. Functional Testing:

Verify the correctness and functionality of all API endpoints as per the API documentation.  
Validate CRUD operations (Create, Read, Update, Delete) and their expected behaviors.  
Validate user authentication and authorization explicitly deployments for protected endpoints.

#### 2. Data Validation Testing:

Ensure that the API correctly validates input data, rejecting invalid requests.  
Verify that the API returns the correct data in response to valid requests.  
Validate the accuracy of data integrity in standard or invalid response data.

#### 3. Error Handling Testing:

Verify that appropriate error codes and messages are returned for invalid requests.  
Validate the API ability to handle unexpected errors gracefully.

#### 4. Security Testing:

Identify potential security vulnerabilities in the API (e.g., SQL injection, XXE).  
Validate that all sensitive data is not exposed in responses.

#### 5. Load Testing:

Measure the API response time under normal and peak loads.  
Validate the API's ability to handle unexpected errors gracefully.

#### 6. Stress Testing:

Assess the API under extreme stress conditions to identify potential failure points.

#### 7. Compatibility Testing:

Test the API on different platforms, browsers, and devices to ensure cross-compatibility.

#### 8. Documentation Review:

Assess the clarity, completeness, and accuracy of the API documentation.  
Validate the API documentation is in sync with the actual API behaviors.

#### 9. Load Testing:

Evaluate the API behavior under high-concurrency user loads to ensure stability.

#### 10. Regression Testing:

Execute regression testing after bug fixes or updates to ensure existing functionality remains intact.

#### 11. Edge Case Testing:

Perform boundary value analysis to identify potential issues.

12. Concurrency Testing:

Assess the API behavior when multiple users attempt to access and modify bookings concurrently.

13. Ad Hoc Testing:

Perform exploratory testing to identify any hidden defects or usability issues.

14. Usability Testing:

Evaluate the API from the viewpoint and ease of use from a developer's perspective.

15. Continuous Integration and Deployment (CI/CD) Testing:

Validate the CI/CD pipeline within the CI/CD pipeline to ensure smooth deployments.

16. Performance Benchmarking:

Implement reporting to track API performance metrics over time.

17. Backup and Recovery Testing:

Perform regular backups and create checks to ensure data integrity.

18. Internationalization Testing:

Test the API with different language settings.

19. Role-based Testing:

Check the API to adhere to role-limiting rules to prevent abuse.

20. Third-Party Integration Testing:

Validate any third-party integrations for smooth functioning.

It's important to note that the nature of the test plan may evolve during the testing process, based on feedback, changing requirements, or discoveries during testing. The scope should be reviewed and adjusted accordingly throughout the testing phases to ensure comprehensive coverage of the Restful Booker API.

## Inclusions

### Create (POST) Operation:-

Send the API to create new bookings using valid input data.

Verify that API handles error responses, are returned for invalid or missing data.

Validate that new bookings are created correctly in the system.

### Read (GET) Operation:-

Verify the API ability to retrieve booking information by (e.g., booking ID, date range, guest name).

Verify that the API returns the correct data in response to valid requests.

Validate the correct handling of non-existent or invalid booking IDs.

### Update (PUT) Operation:-

Test the API's ability to update existing bookings with valid data.

Verify that the API returns handle appropriate error responses.

Validate that the booking data is correctly updated in the system after updates.

### Delete (DELETE) Operation:-

Test the API's ability to delete bookings by providing valid booking IDs.

Verify that the API handles appropriate error responses for invalid input fields.

Validate that the deleted bookings are removed from the system after updates.

### Boundary Testing:-

Test the API's ability to handle bookings by providing valid booking IDs.

Verify that the API validates the boundary values above the boundaries.

#### Concurrency Testing:-

Test the API's ability to handle bookings by providing valid booking IDs.

Validate the behavior of the API when values above the boundaries.

#### Data Validation:-

Test the API response to various data validation scenarios (e.g., invalid characters, data types, mandatory fields).

Verify that the API handles validation errors appropriately.

#### Authentication and Authorization:-

Test the API operations for both authenticated and unauthenticated users.

Validate that appropriate users and messages are returned.

#### Error Monitoring:-

Test the API operations for both authenticated and unauthenticated users.

Validate that appropriate error codes and messages are returned.

#### Security Testing:-

Test for security vulnerabilities during CRUD operations (e.g., SQL injection, XXE).

Validate that all sensitive data is not exposed in responses.

#### Performance Testing:-

Measure the API response time under normal and peak loads.

Showcase the throughput and scalability of the API.

#### Integration Testing:-

Verify the interaction and data consistency between CRUD operations and other API components.

#### Regression Testing:-

Perform regression tests after changes or updates to ensure existing CRUD functionalities remain intact.

#### Documentation Review:-

Assess the accuracy of the API documentation related to CRUD operations.

#### I and E Testing:-

Evaluate the API between and performance during CRUD operations under high assessment user loads.

#### Compatibility Testing:-

Test the CRUD operations on different platforms, browsers, and devices.

#### Usability Testing:-

Evaluate the ease of using CRUD functionalities from a developer's perspective.

#### Continuous Integration and Deployment (CI/CD) Testing:-

Validate the CRUD operations within the CI/CD pipeline to ensure smooth deployments.

#### Role-Limiting Testing:-

Check the API's adherence to role-limiting rules for CRUD operations.

Validate state backup and recovery procedures for CRUD related data.

restful-booker

#### Test Environments

The operating systems that will be used for testing, such as Windows 10, Mac OS, or Linux.

The browsers and versions that will be used for testing, such as Google Chrome, Mozilla Firefox, or Microsoft Edge.

The devices types and screen sizes that will be used for testing, such as desktop computers, laptops, tablets, and smartphones.

The network connectivity and bandwidth that will be available for testing, such as Wi-Fi, 3G, or wired connections.

The testing team and software requirements for running the test cases, such as a specific processor, memory or storage capacity.

Name Role

QA

[restful-booker.herokuapp.com/apidoc/index.html](https://restful-booker.herokuapp.com/apidoc/index.html)

Key Prod

[restful-booker.herokuapp.com/apidoc/index.html](https://restful-booker.herokuapp.com/apidoc/index.html)

Windows 10 - Chrome, Firefox via Edge

Mac - OS X - Safari Browser

Android Mobile OS - Rabbit

iPhone Mobile OS - Rabbit

## Defect Reporting Procedure

This outlines for identifying a defect, such as a deviation from the requirements, save opportunities issues, or functional deviations.

The steps for reporting a defect, such as using a designated template, providing detailed information about the issue, and including screenshots or logs.

The process for triaging and prioritizing defects, such as assigning severity and priority levels, and assigning them to the appropriate team members for investigation and resolution.

The tools and systems that will be used to measure the effectiveness of the defect reporting process, track on them, get updates and for managing defects, such as a defect tracking software or a project management tool.

The roles and responsibilities of the team members involved in the defect reporting process, such as testers, developers, and the test lead.

The communication channels and frequencies for updating relevant stakeholders on the progress and status of defects.

## Defect Priorities POC

Total Functional

Scratch

Backend

Scratch

Core Ops

Projects

Tools:- JIRA

## Test Strategy

The first step is to create test scenarios and test cases for the various features in the Booker

Application.

While developing test cases, we will follow a number of test design techniques:

Equivalence Value Analysis

Boundary Value Analysis

Decision Table Testing

State Transition Testing

Use Case Testing

We will also take into consideration in writing Test Cases by applying the below:

Error Guessing

Exploratory Testing

We will prioritize the Test Cases.

Step 2: Our testing procedure when we receive a request for testing:

First, well-considered individual testing to test for various and test important functionalities of the application are working.

We report the finds in bug-tracking tools and will work for the Booker Testing role in the stable editor performing or simply testing the application functionalities.

Once we receive a stable build which passes Smoke Testing, we perform or start testing using the following plan for the stable builds:

Multiple Test Resources will be testing the same Application on Multiple Requested Environment simultaneously.

Step 3: We will follow the below best practices to make our testing better:

Contract Driven Testing: We will be performing Testing as per the context of the given application.

Shift Left Testing: We will start testing from the beginning stages of the development build instead of testing for stable builds.

Exploratory Testing: Using our expertise we will perform Exploratory testing paper from the initial iteration of the first sprint.

End to End Post Plan Testing: We will test the end to end scenario which enables multiple modules running functionality to end user flows.

## Test Schedule

Following is the test schedule planned for the project:

Task Dates:

Creating Test Plan

Test Cases Creation

Test Case Execution

Summary Reports Submission Date

2 Sprints to Test the Application

## Test Deliverables

The following are to be delivered to the client:

Deliverables Description Target Completion date

Test Plan

Details on the name of the Test Plan, strategy, test schedule, resource requirements, and deliverables

are documented.

Date

Functional Test cases

Test Cases created for the Scope defined.

Date

Defect Reports

Detailed description of the defects identified along with screenshots and steps to reproduce on a daily basis, Bugs by Stage, Bugs by Bugs by Priority, Bugs by Functional Area and Bugs by Priority.

Date

Entry and Exit Criteria

Entry and Exit Criteria will be set for every phase of Software Testing Life Cycle.

Entry Criteria:

Once the testing team ensures the Requirements, Documents, or details about the Project

Exit Criteria:

List of Requirements are captured and understood by the Testing team.

Doubts are cleared.

Test Execution

Entry Criteria:

Test Scenarios and Test Cases Documents are signed off by the Client.

Application is ready for Testing.

Exit Criteria:

Test Case Reports, Defect Reports are ready.

Test Closure

Entry Criteria:

Test Case Reports, Defect Reports are ready.

Exit Criteria:

Test Summary Reports.

Tools

The following are the list of Tools we will be using in this Project:

JIRA Bug Tracking Tool

Mind map Tool

Word and Excel documents

Risks and Mitigations

Following the list of risks as much as possible and the ways to mitigate them:

Risk: Stable URL is not working

Mitigation: Solution Planning

Risk: Extra time for Testing.

Mitigation: Resources will work on local API calls.

Risk: Less time for Testing.

Mitigation: Resources will work on local API calls.

Mitigation: Ramp up the resources based on the Client needs dynamically.

#### Approvals

There will be several different types of documents for Client Approval like below:

Test Plan

Test Architecture

Test Cases

Test Reports

Testing will continue in the same layers once these approvals are done.